TinyML Compression and Feature Engineering for Real-Time IMU Gesture Recognition on Arduino Nano 33 BLE

Mohammadreza Zamani
Università di Trento
Trento, Italy

Asal Abbas Nejad Fard
Università di Trento
Trento, Italy

ABSTRACT

This paper investigates efficient TinyML solutions for real-time inertial measurement unit (IMU)-based gesture recognition on a resource-constrained microcontroller.

Two optimization strategies are evaluated: neural-network compression and feature engineering. First, convolutional neural networks are optimized using lightweight architectures and INT8 quantization. Second, handcrafted RMS features are investigated as an alternative representation for reducing computational complexity.

The proposed RMS Tiny INT8 model achieves 100% accuracy on the fixed 32-window test set while requiring only 284 parameters, 256 neural-network MAC operations, and a 3.414 KB TensorFlow Lite model size.

The final model is deployed on an Arduino Nano 33 BLE using TensorFlow Lite Micro and achieves 0.165 ms neural-network inference latency and 0.346 ms total processing time.

KEYWORDS

TinyML, IMU, Gesture Recognition, TensorFlow Lite Micro, INT8 Quantization, Arduino Nano 33 BLE

1 INTRODUCTION

Tiny Machine Learning (TinyML) enables machine-learning inference directly on embedded devices with limited memory and computational resources. Recent work has demonstrated the potential of on-device time-series classification and human activity recognition on resource-constrained embedded platforms [2, 5].

However, deploying neural networks on microcontrollers requires a careful trade-off between model complexity, memory footprint, and inference latency. Efficient embedded inference and systematic hardware benchmarking are therefore important parts of TinyML system design [1, 3].

This work investigates two approaches for efficient embedded gesture recognition:

- Neural-network compression using lightweight CNN architectures and INT8 quantization.
- Feature engineering using low-dimensional RMS representations combined with compact neural networks.

The main contributions are:

- Evaluation of CNN compression techniques for embedded IMU classification.
- Comparison of raw-signal and feature-based representations.
- Development of an INT8 TinyML model suitable for microcontroller deployment.
- Physical hardware benchmarking on an Arduino Nano 33 BLE.

2 SYSTEM OVERVIEW

The complete system pipeline is:

6-Axis IMU Sensor → Window Segmentation → Feature Extraction / CNN Processing
→ INT8 TinyML Model → Arduino Nano 33 BLE → Gesture Prediction

The system processes six IMU channels:

- Accelerometer: aX, aY, aZ
- Gyroscope: gX, gY, gZ

3 HARDWARE AND SOFTWARE SETUP

The target embedded platform and software stack include:

- Arduino Nano 33 BLE
- TensorFlow Lite Micro
- TensorFlow Lite INT8 quantization
- Embedded C/C++ model deployment

The objective is to perform real-time gesture recognition directly on the microcontroller without cloud processing.

4 DATASET AND EXPERIMENTAL SETUP

The dataset consists of six-axis IMU measurements acquired from three accelerometer channels (aX, aY, aZ) and three gyroscope channels (gX, gY, gZ).

Each sample is segmented into a fixed window of

$$128 \times 6 = 768$$

raw sensor values.

The dataset contains 320 windows distributed equally across four gesture classes, with 80 windows per class.

Table 1: Gesture Classes

Class ID	Gesture
0	circle
1	left_right
2	rest
3	up_down

Table 2: Study 1 Offline Model Comparison

Model	Params.	NN MACs	Size (KB)	Acc.
Standard CNN FP32	2660	160320	15.121	100%
Standard CNN INT8	2660	160320	9.797	100%
Depthwise CNN FP32	1330	52544	10.824	100%
Depthwise CNN INT8	1330	52544	10.125	100%

A fixed stratified 80/10/10 split is used for the main experiments:

- Training: 256 windows
- Validation: 32 windows
- Test: 32 windows

The test set therefore contains eight windows from each gesture class.

All normalization statistics are estimated using the training set only and are then applied unchanged to the validation and test sets. This prevents information from the evaluation subsets from leaking into the training pipeline.

5 STUDY 1: NEURAL NETWORK COMPRESSION

The first study investigates the effect of neural-network architecture and INT8 post-training quantization on the efficiency of raw-signal gesture classification.

Two convolutional architectures are evaluated:

- Standard CNN
- Depthwise-separable CNN

Depthwise-separable convolutions are commonly used to reduce computational cost in resource-constrained sensor-classification systems [4].

Both architectures operate directly on the normalized 128×6 IMU window. Each architecture is evaluated in both FP32 and fully quantized INT8 TensorFlow Lite formats.

The same training, validation, and test partitions are used for all models to ensure a controlled comparison.

The Standard CNN contains 2,660 trainable parameters and requires 160,320 neural-network MAC operations per inference.

Replacing standard convolutions with depthwise-separable convolutions reduces the parameter count to 1,330 and the MAC count to 52,544. This corresponds to approximately a 50% reduction in parameters and a 67.2% reduction in neural-network MAC operations.

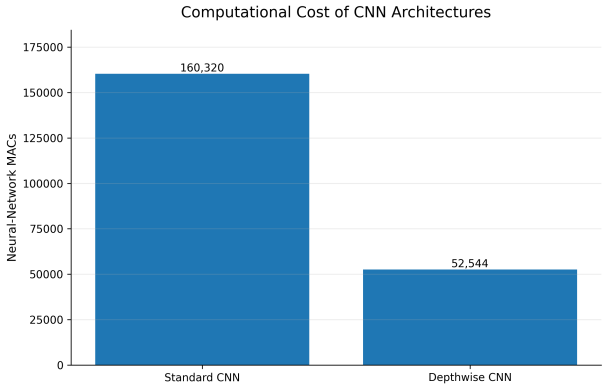

Figure 1: Neural-network MAC comparison of the Standard and Depthwise CNN architectures.

Table 3: Study 1 Arduino INT8 Deployment Results

Metric	Standard CNN	Depthwise CNN
Flash (bytes)	194408	205424
Tensor Arena (bytes)	6644	7156
Inference (ms)	25.370	16.880
Total Processing (ms)	26.760	18.330

INT8 quantization reduces the Standard CNN TFLite model size from 15.121 KB to 9.797 KB. The reduction is smaller for the Depthwise CNN because its parameter set is already compact and fixed TensorFlow Lite metadata represents a larger fraction of the final model file.

Although the Depthwise CNN requires fewer parameters and MAC operations, its measured Flash and Tensor Arena usage are slightly higher than those of the Standard CNN.

Nevertheless, the Depthwise CNN reduces measured inference latency from 25.370 ms to 16.880 ms, corresponding to approximately a 33.5% reduction.

6 STUDY 2: FEATURE ENGINEERING

The second study investigates whether feature engineering can provide a more efficient representation for TinyML gesture classification.

Four input representations are evaluated:

- Raw signal
- Root Mean Square (RMS)
- Fast Fourier Transform (FFT)
- Spectral features

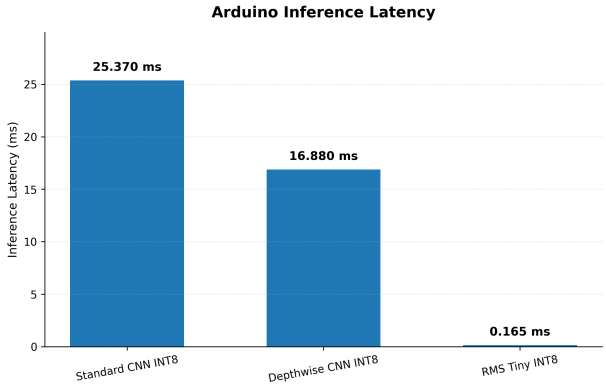

Figure 2: Measured inference latency on the Arduino Nano 33 BLE for the three final INT8 models.

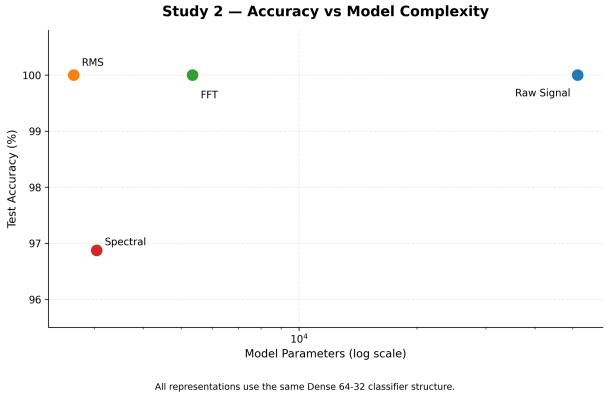

Figure 3: Test accuracy versus parameter count for the four controlled feature representations.

Table 4: Study 2 Feature Representation Comparison

Representation	Features	Params.	Acc.	Macro F1
Raw Signal	768	51428	100.000%	1.0000
RMS	6	2660	100.000%	1.0000
FFT	48	5348	100.000%	1.0000
Spectral	12	3044	96.875%	0.9686

To isolate the effect of the input representation, all four representations are evaluated using the same classifier structure:

$$\text{Input} \rightarrow \text{Dense}(64) \rightarrow \text{Dense}(32) \rightarrow \text{Dense}(4)$$

The only difference between experiments is the dimensionality and content of the input representation.

The raw representation uses all 768 values from the 128×6 IMU window. In contrast, RMS reduces each window to only six features, one for each IMU channel.

Despite this substantial dimensionality reduction, the RMS representation achieves the same test accuracy and Macro F1 score as both the raw signal and FFT representations on the current test set.

The spectral representation uses only 12 features but produces a slightly lower test accuracy of 96.875%, indicating that the additional spectral compression removes information useful for classification in this experiment.

The controlled comparison therefore identifies RMS as the most efficient representation among those achieving 100% test accuracy: it reduces the input dimensionality from 768 raw values to only six features and reduces the classifier parameter count from 51,428 to 2,660.

Table 5: Final RMS Tiny Model Characteristics

Metric	Value
Input features	6
Parameters	284
NN MACs	256
FP32 TFLite size	3.227 KB
INT8 TFLite size	3.414 KB
INT8 test accuracy	100.000%
INT8 Macro F1	1.0000

7 FINAL RMS TINY INT8 MODEL

The controlled feature-engineering study identified RMS as the most efficient representation among the alternatives that achieved 100% test accuracy.

For each IMU channel, the RMS feature is computed over the 128-sample window as

$$\text{RMS}_c = \sqrt{\frac{1}{N}\sum_{n=1}^{N} x_c[n]^2}, \tag{1}$$

where $N = 128$ and c denotes one of the six IMU channels.

Consequently, each 128×6 raw window is reduced to only six RMS values.

The six RMS features are standardized using mean and scale parameters estimated exclusively from the training set. The resulting feature vector is then provided to the optimized classifier.

The final architecture is

$$6 \text{ RMS Features} \rightarrow \text{Dense}(16) \rightarrow \text{Dense}(8) \rightarrow \text{Softmax}(4)$$

This architecture contains only 284 trainable parameters and requires 256 neural-network MAC operations per inference.

Both the FP32 and INT8 versions preserve the classification performance observed on the current test set.

Interestingly, the INT8 TFLite file is slightly larger than the FP32 file for this extremely small network. This occurs because fixed TensorFlow Lite quantization metadata represents a non-negligible fraction of the total file size when the number of model parameters is very small.

Therefore, INT8 quantization is selected primarily for integer microcontroller deployment rather than for additional file-size reduction in this particular model.

The reported 256 MAC operations refer only to the neural-network classifier and do not include the RMS feature-extraction cost.

8 EMBEDDED DEPLOYMENT RESULTS

The three final INT8 models were deployed on the Arduino Nano 33 BLE using TensorFlow Lite Micro and evaluated using physical on-device measurements.

The deployment comparison includes:

- Standard CNN INT8
- Depthwise CNN INT8
- RMS Tiny INT8

For the CNN models, the preprocessing stage consists primarily of raw-signal normalization. For the RMS Tiny model, the preprocessing stage additionally includes RMS feature extraction and feature standardization.

The Standard CNN requires 25.370 ms for neural-network inference, while the Depthwise CNN reduces the measured inference latency to 16.880 ms.

The RMS Tiny INT8 model provides a substantially larger reduction, requiring only 0.165 ms for neural-network inference and 0.346 ms for the complete processing pipeline.

Compared with the Standard CNN INT8 model, RMS Tiny reduces neural-network inference latency by approximately 99.35% and total processing time by approximately 98.71%.

Compared with the Depthwise CNN INT8 model, the corresponding reductions are approximately 99.02% and 98.11%, respectively.

The Tensor Arena requirement is reduced from 6,644 bytes for the Standard CNN and 7,156 bytes for the Depthwise CNN to only 948 bytes for RMS Tiny.

The measured Flash usage of RMS Tiny is 165,960 bytes, representing a reduction of approximately 14.6% relative to the Standard CNN deployment and 19.2% relative to the Depthwise CNN deployment.

The final RMS Tiny model therefore provides the smallest neural-network complexity, the smallest Tensor Arena requirement, and the lowest measured end-to-end processing latency among the three deployed alternatives.

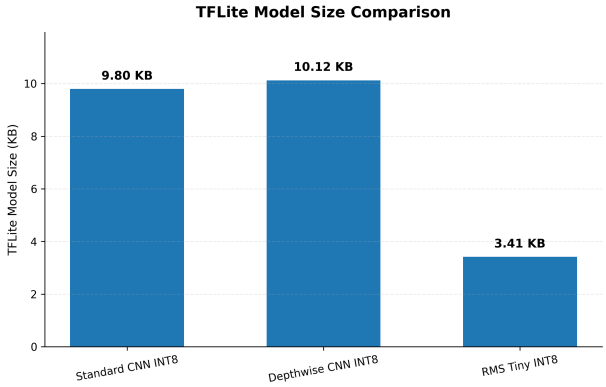

Figure 4: TFLite model size comparison for the three final INT8 deployment models.

These results are consistent with recent work emphasizing the importance of physical latency and resource measurements when evaluating embedded AI systems [1, 3].

9 DISCUSSION

The experimental results show that TinyML efficiency can be improved through both neural-network compression and feature engineering.

The Depthwise CNN substantially reduces MAC operations and inference latency compared with the Standard CNN, although its measured Flash and Tensor Arena requirements are slightly higher.

Feature engineering provides the largest overall reduction in this study. RMS reduces each input window from 768 raw IMU values to only six features while maintaining 100% accuracy on the current test set. This enables the final RMS Tiny INT8 model to use only 284 parameters and 256 neural-network MACs.

On the Arduino Nano 33 BLE, RMS Tiny requires 0.165 ms for neural-network inference and 0.346 ms for the complete processing pipeline.

The reported 100% accuracy should nevertheless be interpreted in the context of the current 32-window test set. Larger datasets, multiple users, and independent recording sessions are required for a broader evaluation of real-world generalization.

10 CONCLUSION

This work evaluated neural-network compression and feature engineering for TinyML-based IMU gesture recognition.

Depthwise CNN reduced the computational cost of the original CNN, while RMS feature engineering provided the largest overall efficiency improvement.

The final RMS Tiny INT8 model achieved 100% accuracy on the current test set using only 284 parameters and 256

Table 6: Final Arduino Nano 33 BLE Deployment Comparison

Model	Flash (B)	Tensor Arena (B)	Preprocess / Feature (ms)	Inference (ms)	Total Processing (ms)
Standard CNN INT8	194408	6644	1.390	25.370	26.760
Depthwise CNN INT8	205424	7156	1.450	16.880	18.330
RMS Tiny INT8	165960	948	0.181	0.165	0.346

neural-network MACs. On the Arduino Nano 33 BLE, it required 0.165 ms inference time and 0.346 ms total processing time.

These results show that lightweight feature engineering combined with compact neural networks is a promising approach for efficient TinyML deployment on resource-constrained devices.

REFERENCES

[1] Leonardo Lucio Custode, Pietro Farina, Eren Yildiz, Renan Beran Kilic, Kasim Sinan Yildirim, and Giovanni Iacca. 2024. Fast-Inf: Ultra-Fast Embedded Intelligence on the Batteryless Edge. In *Proceedings of the 22nd ACM Conference on Embedded Networked Sensor Systems (SenSys '24)*. ACM, 239–252. https://doi.org/10.1145/3666025.3699335

[2] Imola Fodor, Roberto Lorenzon, Gilberto Pin, Stefano Antonello, and Kasim Sinan Yildirim. 2025. Embedded-AI Based Fault Detection in Domestic Dishwashers: A Tale of On-Device Time Series Deep Classification. *IFAC-PapersOnLine* 59, 26 (2025), 371–376. https://doi.org/10.1016/j.ifacol.2025.12.063

[3] Mohammad Amin Hasanpour, Mikkel Kirkegaard, and Xenofon Fafoutis. 2025. EdgeMark: An Automation and Benchmarking System for Embedded Artificial Intelligence Tools. *Journal of Systems Architecture* 167 (2025), 103488. https://doi.org/10.1016/j.sysarc.2025.103488

[4] Imran Hossan, Mst. Nusratul Jannat Mary, and Mohammod Abdul Motin. 2025. TinyHAR-Net: Design and Implementation of a TinyML-Based Human Activity Recognition Framework on STM32. *IEEE Embedded Systems Letters* (2025). https://doi.org/10.1109/LES.2025.3639944

[5] Haotian Zhou, Xiujun Zhang, Yu Feng, Tongda Zhang, and Lijuan Xiong. 2025. Efficient Human Activity Recognition on Edge Devices Using DeepConv LSTM Architectures. *Scientific Reports* 15 (2025), 13830. https://doi.org/10.1038/s41598-025-98571-2